

Capítulo 5

Modelo de crecimiento urbano

Este capítulo muestra una visión exhaustiva del modelo citado a lo largo de todo este trabajo como WoE-A, se presentan las decisiones explícitas de cada componente del modelo, preprocesamiento y validación. Si bien el segundo capítulo del presente trabajo da una definición a cada concepto utilizado, no hay una argumentación de por qué se utiliza cada una de las componentes del flujo completo que al trabajo presente le compete. Se inicia, como ya el lector debe saber, con la definición del flujo de preprocesamiento de datos, seguido por el modelo que consume los datos preprocesados para calcular la matriz de transiciones históricas, seguido por la definición del autómata celular probabilístico y finalmente la validación del modelo.

5.1. Preprocesamiento de datos.

El preprocesamiento de datos, para este trabajo sienta la verdad que el modelo WoE-AC utiliza para predecir la dinámica urban. Por ende, se debe entender el preprocesamiento de datos como el proceso que convierte las capturas RGB en mapas binarios urbano/no-urbano (pipeline LBP + K-Means + SVM); en el capítulo ?? en particular en la figura ?? se muestra el flujo completo de preprocesamiento de datos, primero: se obtiene un tensor RGB de $1,792 \times 3,024 \times 3$ píxeles, esto constituye la información cruda de entrada al flujo.

En el siguiente paso, se realiza una transformacion de este arreglo de $1,792 \times 3,024 \times 3$ píxeles a un arreglo de $1,792 \times 3,024 \times 23$ píxeles. Este paso conlleva la primer decisión critica en la definición del flujo, pues en vez de utilizar índices espectrales, se decide obtener 23 features por celda. Esto tiene una reazón de ser: las capturas disponibles son RGB, de tres bandas visibles, NDVI requiere NIR; NDWI requiere NIR o SWIR; NDBI requiere SWIR. Ninguno de esos índices es computable a partir de RGB puro, eso elimina la clase mas común de clasificadores

de cobertura de suelo que usa la literatura. Esto tiene una consecuencia directa, con solo bandas visibles, los píxeles urbanos (techo gris, asfalto) y los de suelo desnudo (arcilla, tierra agrícola seca) tienen un parecido espectral RGB prácticamente indistinguibles. La solución a este problema consiste en extender la representación por pixel con 23 features definidas en las tablas ??, ?? y ?. El objetivo es codificar la textura (LBP, Sobel, entropía, contraste) y espacios de color perceptuales (LAB, HSV). El tejido urbano tiene un patrón espacial regular, cuadrícula de calles, alineación de techados; que la vegetación y el suelo agrícola no tienen. LBP captura exactamente esa regularidad; es un descriptor diseñado para distinguir texturas y ha sido validado en clasificación de imágenes de satélite de baja resolución espectral.

Sin embargo,

La metodología propuesta integra dos paradigmas complementarios. El **Weight of Evidence (WoE)** transforma las variables espaciales en evidencia probabilística cuantificada a partir de patrones históricos observados. Los **Autómatas Celulares (AC)** proporcionan el mecanismo de simulación espacialmente explícita de dinámicas urbanas, incorporando las evidencias WoE en reglas de transición probabilísticas.

5.1.1. Principios de diseño

En respuesta a las limitaciones identificadas en la revisión de la literatura (Capítulo ??) —en particular la validación temporal acotada, el uso de métricas parciales y la escasez de estudios en ciudades intermedias latinoamericanas—, el modelo propuesto adopta un esquema de validación múltiple, emplea métricas complementarias (FoM, Kappa, IoU, F1) y se contextualiza en la Zona Metropolitana de Querétaro.

La metodología se sostiene en tres principios:

Evidencia empírica el Weight of Evidence aporta una cuantificación objetiva de las relaciones espaciales observadas históricamente, mediante el análisis estadístico de las transiciones urbanas pasadas.

Simulación espacial explícita los Autómatas Celulares capturan dinámicas espaciales complejas, efectos de vecindario y patrones emergentes a partir de reglas locales de transición.

Validación temporal el protocolo de evaluación se aplica sobre múltiples períodos independientes, para estimar la robustez predictiva del modelo.

5.1.2. Flujo Metodológico

El proceso metodológico sigue una secuencia estructurada en cinco fases.

1. **Preparación de datos:** construcción de mapas urbanos binarios anuales (1984–2020) a partir de las capturas RGB, y cálculo de las siete variables espaciales derivadas (distancias, densidades de vecindad a tres escalas, fragmentación local, gradiente y tamaño de cluster).
2. **Cálculo WoE:** cuantificación de la evidencia espacial mediante el análisis agregado (*pooling*) de las transiciones históricas anuales del período 1984–2010.
3. **Configuración del Autómata Celular:** definición de las reglas de transición que combinan la evidencia WoE con la influencia vecinal y el decaimiento espacial exponencial.
4. **Calibración de parámetros:** descrita en el Capítulo ???. Combina exploración sistemática en cuadrícula y ajuste manual del umbral de transición. Los pesos efectivos quedan registrados en `data/processed/quinquenal_best_config.json`; los parámetros numéricos heredados (`neighbor_weight`, `distance_weight`) provienen de un experimento exploratorio previo y se adoptan aquí como valores fijos (ver nota metodológica en la Sección ??).
5. **Validación quinquenal múltiple:** evaluación del desempeño predictivo en cinco ventanas temporales independientes (2011–2016, 2012–2017, 2013–2018, 2014–2019 y 2015–2020), con métricas espaciales estándar (Accuracy, IoU, Precision, Recall, F1, Kappa y FoM). Su presentación detallada también se realiza en el Capítulo ??.

5.2. Preprocesamiento de la serie de imágenes de entrada

Antes de aplicar el modelo WoE-AC, las **capturas RGB** anuales (obtenidas con Google Earth e imagen histórica, según el Capítulo ??) se transforman en representaciones binarias urbano/no-urbano mediante el pipeline LBP+K-Means+SVM. Esta sección describe ese procedimiento y su rol en la construcción de la serie temporal de 37 años que alimenta el análisis de transiciones históricas.

5.2.1. Transformación a mapas binarios urbano/no-urbano

El primer paso consiste en convertir las matrices RGB —tres canales por píxel, más los descriptores derivados en distintos espacios de color e índices aproximados a partir de RGB— en mapas binarios que representen la cobertura urbana en sentido amplio. Este preprocesamiento sienta las bases del análisis WoE-AC posterior.

Extracción de características con Local Binary Patterns

Se emplea **Local Binary Pattern (LBP)** (Ojala2002) como descriptor de textura para capturar patrones espaciales característicos de áreas urbanas en las **capturas RGB** de la serie. LBP

puede ayudar a separar texturas asociadas a tejido urbano (edificaciones, calles, infraestructura) de texturas más propias de vegetación o suelo descubierto, **en el marco de la información disponible** (tres bandas y sus derivados).

La idea general del LBP codifica contraste local entre el píxel central y vecinos sobre una circunferencia; la forma clásica de 8 vecinos en ventana 3×3 sirve como referencia conceptual (Ojala2002). En la implementación del repositorio (`skimage.feature.local_binary_pattern`), el LBP se calcula sobre la **imagen en escala de grises** con radio $R = 3$ y $P = 8R = 24$ puntos de muestreo sobre la circunferencia, método uniform, tal como en `tesis_ac/pipeline/feature\_extraction.py` (`FeatureExtractor.extract_texture_features`). Esta configuración es la que condiciona el resto del pipeline.

$$\text{LBP}_{R,P}(x, y) \quad \text{con } R = 3, P = 24, \text{ method} = \text{uniform} \quad (5.1)$$

Clasificación binaria urbano/no-urbano

Las características LBP extraídas alimentan un clasificador binario híbrido que combina clustering no supervisado (K-Means) con clasificación supervisada (SVM), buscando **coherencia interna** y estabilidad en la serie temporal, sin pretender óptimo global respecto a una verdad terreno independiente.

Pipeline de Clasificación Híbrida:

1. **Extracción de características:** La clase `FeatureExtractor` (`tesis_ac/pipeline/feature\_extraction.py`), con `extract_full=True`, construye por píxel un vector $f(x, y) \in \mathbb{R}^{23}$ al concatenar:
 - **Básicas (8):** canales R, G, B; intensidad media; brillo; desviaciones locales R/G/B (`rank.mean` en ventana 5×5).
 - **Textura (6):** Sobel horizontal/vertical; magnitud del gradiente; LBP uniforme ($R = 3$, $P = 24$) sobre escala de grises; entropía local; contraste local.
 - **Espectrales / color (9):** NDVI aproximado, exceso de verde y de rojo; componentes L,a,b en LAB; H,S,V en HSV.

Esta lista coincide con el orden consolidado en `consolidate_features`.

2. **Estandarización y reducción dimensional con PCA:** Los 23 descriptores se normalizan con `StandardScaler` mediante la clase `FeaturePreprocessor` (`n_components=8`), reduciendo de 23 a 8 dimensiones antes del clustering y el SVM.
3. **Clustering K-Means** ($k \in \{2, 3, 4\}$): la clase `ImageClusterer` (`tesis_ac/pipeline/`

`clustering.py`) entrena K-Means para cada valor de k y las pseudoetiquetas resultantes alimentan al SVM. La selección del mejor k se realiza con los criterios implementados en `get_best_clustering / get_best_classifier` (típicamente inercia y desempeño del clasificador). El algoritmo minimiza la inercia:

$$\min_C \sum_{i=1}^k \sum_{f \in C_i} \|f - \mu_i\|^2 \quad (5.2)$$

donde $\{C_1, \dots, C_k\}$ son los k clusters y $\{\mu_1, \dots, \mu_k\}$ sus centroides. Las etiquetas K-Means proporcionan así un pseudo *ground truth* para entrenar el SVM.

4. **Clasificación SVM con kernel lineal:** Se entrena un Support Vector Machine usando las etiquetas K-Means como supervisión. En la formulación estándar de margen máximo, el SVM determina el hiperplano separador:

$$\max_{\mathbf{w}, b} \frac{2}{\|\mathbf{w}\|} \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{f}_i + b) \geq 1, \quad \forall i \quad (5.3)$$

donde $y_i \in \{-1, +1\}$ son las etiquetas K-Means, y $\mathbf{w}, b$ definen el hiperplano de separación. El código usa `sklearn.svm.SVC(kernel='linear')` vía la clase `ImageClassifier`.

5. **Predicción final:** El clasificador SVM entrenado predice la clase binaria para todos los píxeles de la imagen:

Formalización matemática unificada:

Sea $I(x, y) \in \mathbb{R}^{H \times W \times 3}$ la imagen RGB de entrada. El mapa binario urbano $U(x, y) \in \{0, 1\}$ se obtiene mediante:

$$U(x, y) = \text{SVM}(\text{PCA}(\mathbf{f}(x, y))) \quad (5.4)$$

donde $\mathbf{f}(x, y) \in \mathbb{R}^{23}$ es el vector de características del píxel (`FeatureExtractor`), PCA reduce a 8 componentes (`FeaturePreprocessor`), y el SVM opera sobre el espacio PCA con etiquetas derivadas de K-Means (`SatelliteImageProcessor`).

Validación de la clasificación

Validación interna (*train/test split*): la calidad del clasificador SVM se evalúa mediante validación cruzada estándar durante el entrenamiento, con las siguientes reglas:

- **División estratificada 80/20:** se reserva el 20 % de los píxeles para el conjunto de prueba,

manteniendo proporciones de clase equilibradas.

- **Evaluación en conjunto de prueba independiente:** el SVM entrenado con el 80 % predice sobre el 20 % restante, no visto durante el entrenamiento.
- **Métricas reportadas:** *Accuracy* (fracción de píxeles correctamente clasificados), *Precision* y *Recall* por clase (falsos positivos y falsos negativos), y *Classification Report* completo (incluye F1-score y soporte por clase).

En los experimentos realizados sobre Querétaro, el método híbrido LBP+K-Means+SVM arrojó *accuracy* interno del orden de 88–94 % en el conjunto de prueba; ello caracteriza la consistencia interna del *pipeline* y no pretende establecer superioridad frente a otros enfoques no incluidos en esta comparación.

Limitación reconocida. Esta validación evalúa la *consistencia interna* del *pipeline* —qué tan bien reproduce el SVM las pseudoetiquetas de K-Means—, no la exactitud respecto a una verdad terreno externa. Una validación completa requeriría: mapas cartográficos oficiales de uso de suelo urbano para todos los años (no disponibles), imágenes de muy alta resolución con digitalización manual, o muestras de campo geolocalizadas con GPS (costoso para series temporales largas).

Como evidencia indirecta de la calidad de los mapas binarios, la validación temporal del AC completo (Capítulo ??) muestra que el modelo, calibrado con mapas históricos, produce predicciones razonables en períodos futuros bajo la misma definición operativa de la clase urbana.

Serie Temporal de Mapas Binarios

El proceso se repite para cada año disponible en la serie temporal anual (1984–2020 en el caso de Querétaro), generando una colección de mapas binarios $\{U_t(x, y)\}_{t=1984}^{2020}$ que resume la evolución de la huella urbana según la clasificación adoptada.

Esta serie temporal binaria constituye el insumo fundamental para:

1. Cálculo de transiciones urbanas $\Delta U_t = U_{t+1} - U_t$
2. Derivación de variables espaciales (distancias, densidades, gradientes)
3. Entrenamiento del modelo WoE mediante pooling temporal de transiciones
4. Validación temporal del modelo AC en períodos independientes

5.2.2. Implementación Computacional

El pipeline de preprocesamiento se implementa en Python en el paquete `tesis_ac/pipeline/`. La clase `SatelliteImagePipeline` (`main_pipeline.py`) encadena `FeatureExtractor.extract_all_features`, `FeaturePreprocessor.fit_transform` y `SatelliteImageProcessor.fit`, que a su vez usa `ImageClusterer` e `ImageClassifier` (`clustering.py`). El fragmento siguiente resume la interfaz pública; los detalles numéricos (23 descriptores, LBP $R=3$, $P=24$) están en el código citado.

```
1 from pathlib import Path
2 from tesis_ac.pipeline.main_pipeline import SatelliteImagePipeline
3
4 pipeline = SatelliteImagePipeline(
5     n_clusters_list=[2, 3, 4],
6     n_pca_components=8,
7     extract_full_features=True,
8     svm_kernel="linear",
9 )
10 # Por cada captura PNG RGB homogeneizada (p. ej. 1792 x 3024):
11 pipeline.process_image(str(Path("ruta") / "YYYY.png"))
12 # Salida: etiquetas por píxel; la binarización urbana/no urbano sigue la
13 # lógica
14 # del proyecto (post-procesado y estandarización hacia standardized_maps/).
```

Listing 5.1: Orquestación del clasificador RGB→binario (`tesis_ac/pipeline/main_pipeline.py`)

Optimizaciones computacionales:

- **Vectorización:** operaciones matriciales aceleradas con BLAS/LAPACK a través de NumPy.
- **PCA:** reduce de 23 características a 8 componentes principales, acelerando K-Means y SVM.
- **Muestreo para el entrenamiento del SVM:** `SatelliteImageProcessor.fit` permite limitar el tamaño de muestra (`sample_size`, por defecto del orden de 10^3 – 10^4 píxeles) para entrenar el clasificador sin usar todos los píxeles de la imagen.
- **Persistencia:** los mapas binarios estandarizados por año se almacenan como matrices `.npy` en `data/processed/standardized_maps/`, que son las que consumen

`train_woe_pooled.py` y los validadores quinquenales.

Con estas optimizaciones, el procesamiento de una captura homogeneizada típica (del orden de $1,792 \times 3,024$ píxeles y tres canales RGB) puede situarse en el orden de **decenas de segundos** por año en hardware estándar (p. ej. CPU clase i5 y 16 GB RAM), de modo que la serie completa (37 años) resulta factible en una ventana de trabajo de **minutos**, según carga del equipo y parámetros de muestreo del SVM.

5.2.3. Consideraciones Metodológicas

Ventajas del enfoque híbrido no supervisado y supervisado

El método híbrido LBP + K-Means + SVM combina segmentación no supervisada con refinamiento supervisado. Sus ventajas son:

- **No requiere etiquetas manuales:** K-Means genera automáticamente un pseudo *ground truth* para entrenar el SVM.
- **Mayor coherencia interna que una umbralización global simple:** el refinamiento con SVM sobre las pseudoetiquetas suele elevar la *accuracy* frente a líneas base no supervisadas triviales en las mismas pruebas locales, sin sustituir una evaluación con verdad terreno externa.
- **Consistencia temporal:** el mismo *pipeline* se aplica a los 37 años de la serie (1984–2020), sin reetiquetar cada período.
- **Transferibilidad:** una vez entrenado, el modelo SVM puede aplicarse rápidamente a imágenes similares.
- **Eficiencia:** el uso de PCA y de muestreo en el entrenamiento del SVM permite procesar imágenes homogeneizadas del orden de $1,792 \times 3,024$ píxeles en tiempos del orden de diez a veinte segundos por año en CPU estándar (orden de magnitud observado durante el desarrollo).

Las limitaciones reconocidas del enfoque son:

- Menor precisión que la de los métodos supervisados con etiquetas manuales (por ejemplo, *Random Forest* con verdad terreno reporta *accuracy* típica del 93 al 97 % en la literatura).
- Sensibilidad a cambios drásticos de iluminación o de condiciones atmosféricas entre imágenes.
- Dificultad en áreas periurbanas con uso mixto del suelo (transición gradual urbano-rural).

- K-Means asume *clusters* esféricos, lo que puede no capturar formas urbanas irregulares complejas.

Alternativas metodológicas evaluadas

Otros enfoques de clasificación urbana considerados son:

1. **Índices espectrales puros:** NDVI (vegetación), NDBI (construcción) o NDWI (agua). Son más simples (no requieren entrenamiento), pero menos robustos ante variabilidad espectral intra-clase; la *accuracy* típica reportada en la literatura se sitúa en el rango 70–80 %.
2. **Umbralización simple (Otsu no supervisado):** aplica un umbral automático sobre la intensidad o el LBP promedio. Es muy eficiente, pero sensible a iluminación desigual; *accuracy* típica del 80 al 85 %.
3. **Clasificación supervisada pura (*Random Forest*, SVM con etiquetas manuales):** requiere digitalizar manualmente muestras urbanas/no-urbanas para cada imagen (~ 500 a 1,000 píxeles por año), con *accuracy* típica reportada del 90 % o superior. Su costo es prohibitivo para series largas (del orden de decenas de miles de píxeles a etiquetar para 37 años).
4. **Aprendizaje profundo (U-Net, SegNet):** las redes convolucionales semánticas alcanzan *accuracy* muy alta en tareas de segmentación urbana, pero requieren conjuntos de datos etiquetados a gran escala (miles de imágenes anotadas), hardware GPU especializado y tiempos de entrenamiento del orden de horas a días. Por estas razones, su uso es menos accesible en contextos de investigación con recursos modestos (Ma2019).

Justificación del método seleccionado. Se eligió LBP+K-Means+SVM por el balance que ofrece entre cuatro factores: una precisión aceptable (88–94 % de *accuracy* en la validación interna, superior a la del Otsu simple sobre las mismas pruebas y comparable en magnitud a la de enfoques semi-supervisados reportados para contextos similares); un costo computacional moderado (del orden de diez a veinte segundos por imagen en CPU estándar, frente a horas en GPU para los enfoques de aprendizaje profundo); la ausencia de dependencia de etiquetas manuales costosas (K-Means genera pseudoetiquetas de forma automática); y la consistencia temporal de aplicar el mismo *pipeline* a los 37 años de la serie sin necesidad de reetiquetar cada período. Este enfoque resulta adecuado para series temporales largas en las que no se dispone de una verdad terreno externa exhaustiva (Ma2019).

5.3. Componente Weight of Evidence

Con los mapas binarios obtenidos en la sección anterior, el segundo componente del modelo cuantifica la evidencia espacial de transición urbana. A continuación se describen su formulación, el cálculo del Information Value y la implementación computacional empleada.

5.3.1. Fundamento Teórico

Weight of Evidence (**BonhamCarter1994**) cuantifica la fuerza de asociación entre variables predictoras y la ocurrencia de crecimiento urbano. Para cada variable espacial X_i y cada bin j :

$$WoE_{ij} = \ln \left(\frac{P(X_i = j|U = 1)}{P(X_i = j|U = 0)} \right) \quad (5.5)$$

donde $U = 1$ indica crecimiento urbano observado, $U = 0$ indica ausencia de crecimiento urbano, y $P(X_i = j|U = k)$ representa la probabilidad condicional de la clase j dado el estado urbano k .

5.3.2. Information Value

El poder predictivo de cada variable se cuantifica mediante Information Value:

$$IV_i = \sum_{j=1}^{n_i} [P(X_i = j|U = 1) - P(X_i = j|U = 0)] \times WoE_{ij} \quad (5.6)$$

La interpretación estándar de IV establece cinco categorías: valores menores a 0.02 indican variable no predictiva; el rango $0.02 \leq IV < 0.1$ señala poder predictivo débil; valores entre 0.1 y 0.3 corresponden a poder predictivo moderado; el rango $0.3 \leq IV < 0.5$ indica poder predictivo fuerte; finalmente, $IV \geq 0.5$ sugiere poder predictivo muy fuerte, siendo sospechoso de overfitting.

5.3.3. Discretización Optimal

Para variables continuas, se implementa discretización que maximiza IV:

Algorithm 1 Discretización Optimal para WoE

Input: Variable continua X , objetivo binario U , número máximo de bins n_{max} **Output:** Bins óptimos $B = \{b_1, b_2, \dots, b_n\}$ Inicializar con percentiles uniformes $n = 2$ **to** n_{max} Calcular IV para configuración de n bins Evaluar significancia estadística (Chi-cuadrado) IV no mejora significativamente **return** configuración anterior **return** configuración con máximo IV válido

5.3.4. Implementación Computacional

El cálculo WoE se implementa eficientemente:

```

1 def calculate_woe(self, variable_map, urban_change_map):
2     """
3     Calcula Weight of Evidence para variable espacial.
4
5     Parameters:
6     -----
7     variable_map : numpy.ndarray
8         Mapa de variable explicativa
9     urban_change_map : numpy.ndarray
10        Mapa binario de cambio urbano (1=crecimiento, 0=no crecimiento)
11
12    Returns:
13    -----
14    woe_map : numpy.ndarray
15        Mapa de valores WoE transformados
16    iv_score : float
17        Information Value de la variable
18    """
19    # Discretizacion optimal
20    bins = self._optimal_binning(variable_map, urban_change_map)
21
22    # Calculo de WoE por bin
23    woe_values = {}
24    total_positive = np.sum(urban_change_map == 1)
25    total_negative = np.sum(urban_change_map == 0)
26
27    for i, (bin_min, bin_max) in enumerate(bins):

```

```

28     # Mascara para bin actual
29     bin_mask = (variable_map >= bin_min) & (variable_map < bin_max)
30
31     # Casos positivos y negativos en bin
32     pos_in_bin = np.sum((bin_mask) & (urban_change_map == 1))
33     neg_in_bin = np.sum((bin_mask) & (urban_change_map == 0))
34
35     # Probabilidades condicionales
36     p_pos = (pos_in_bin + 0.5) / (total_positive + 1) # Suavizado Laplace
37     p_neg = (neg_in_bin + 0.5) / (total_negative + 1)
38
39     # Weight of Evidence
40     woe_values[i] = np.log(p_pos / p_neg)
41
42     # Transformar mapa original a valores WoE
43     woe_map = self._apply_woe_transformation(variable_map, bins, woe_values)
44
45     # Calcular Information Value
46     iv_score = self._calculate_iv(bins, woe_values, urban_change_map)
47
48     return woe_map, iv_score

```

Listing 5.2: Implementacion de Weight of Evidence

5.4. Componente Autómatas Celulares

Integrando las evidencias WoE calculadas en la sección precedente, el componente de autómatas celulares define la arquitectura de simulación espacial, las reglas de transición probabilísticas y el algoritmo de evolución temporal del modelo.

5.4.1. Arquitectura del autómata

El AC implementado sigue una arquitectura estándar, con las siguientes especificaciones:

Espacio celular cuadrícula regular alineada con los mapas binarios derivados de las capturas RGB (misma resolución espacial de trabajo).

Estados celulares binarios —urbano (1) frente a no-urbano (0)— con un estado adicional RES-

TRICTED (2) para zonas protegidas, cuerpos de agua y áreas con restricciones ambientales. **Vecindario** de Moore con radio 1 (8 celdas adyacentes) y ponderación por distancia. **Reglas de transición** probabilísticas, que combinan la evidencia WoE, la influencia vecinal y un decaimiento espacial exponencial. **Evolución temporal** iteraciones discretas con restricción de tasa máxima de crecimiento por paso.

5.4.2. Variables Espaciales Multi-Escala

El modelo deriva siete variables espaciales directamente del grid urbano histórico, capturando patrones a múltiples escalas:

1. `distance_urban`: Distancia euclidiana a la celda urbana más cercana (transformación `distance_transform_edt`)
2. `neighbor_density_3x3`: Densidad urbana en vecindario inmediato (ventana 3×3)
3. `neighbor_density_5x5`: Densidad urbana en vecindario amplio (ventana 5×5)
4. `neighbor_density_7x7`: Densidad urbana en contexto regional (ventana 7×7)
5. `local_fragmentation`: Heterogeneidad espacial calculada como varianza local (ventana 5×5)
6. `nearest_cluster_size`: Tamaño del cluster urbano contiguo más cercano
7. `urban_gradient`: Magnitud del gradiente espacial de urbanización (filtro Sobel)

Estas variables capturan tanto efectos de proximidad como patrones estructurales del crecimiento urbano sin requerir datos externos adicionales.

5.4.3. Reglas de Transición con Decaimiento Exponencial

Siguiendo la formulación presentada en la Sección ?? del marco teórico, la probabilidad de urbanización para la celda (i, j) se calcula mediante un proceso de tres etapas:

Etapla 1 - Agregación de evidencia: Se combina la influencia vecinal directa con las evidencias WoE de las variables espaciales:

$$S_{i,j} = w_n \cdot I_{neigh}(i, j) + \sum_{k=1}^7 w_k \cdot WoE_k(i, j) \quad (5.7)$$

donde $I_{neigh}(i, j)$ es la proporción de vecinos urbanos en el vecindario Moore, w_n es el peso

de influencia vecinal, w_k son los pesos específicos por variable, y $WoE_k(i, j)$ son los valores transformados de Weight of Evidence para cada variable espacial en la celda (i, j) .

Etapla 2 - Decaimiento espacial exponencial: Se aplica penalización por distancia para concentrar el crecimiento cerca de zonas urbanas existentes:

$$S_{i,j}^{decay} = S_{i,j} \cdot \exp(-\lambda \cdot d_{urban}(i, j)) \quad (5.8)$$

donde $d_{urban}(i, j)$ es la distancia a la zona urbana más cercana y λ es la tasa de decaimiento exponencial. Este factor garantiza que zonas muy alejadas ($d > 20$ celdas) tengan probabilidad prácticamente nula independientemente de otros factores.

Etapla 3 - Conversión probabilística: El score combinado se transforma a probabilidad mediante función logística con componente estocástico:

$$P_{i,j}^{raw} = \sigma(S_{i,j}^{decay}) = \frac{1}{1 + \exp(-S_{i,j}^{decay})} \quad (5.9)$$

$$P_{i,j}^{final} = \begin{cases} 0.1 \cdot P_{i,j}^{raw} + \mathcal{N}(0, \sigma_{stoch}) & \text{si } P_{i,j}^{raw} < \theta_{base} \\ P_{i,j}^{raw} + \mathcal{N}(0, \sigma_{stoch}) & \text{si } P_{i,j}^{raw} \geq \theta_{base} \end{cases} \quad (5.10)$$

donde θ_{base} es el umbral base (típicamente 0,5) y σ_{stoch} controla la variabilidad estocástica (típicamente 0,1). Las probabilidades finales se recortan al intervalo $[0, 1]$.

5.4.4. Cálculo de Influencia Vecinal

La influencia vecinal con ponderación por distancia se calcula como:

$$I_{neigh}(i, j) = \frac{\sum_{(m,n) \in \mathcal{V}_{i,j}} \frac{U_{m,n}^t}{d_{m,n}+1}}{\sum_{(m,n) \in \mathcal{V}_{i,j}} \frac{1}{d_{m,n}+1}} \quad (5.11)$$

donde $\mathcal{V}_{i,j}$ es el vecindario Moore excluyendo la celda central, $U_{m,n}^t \in \{0, 1\}$ es el estado urbano, y $d_{m,n} = \max(|i - m|, |j - n|)$ es la distancia Chebyshev. Esta formulación da mayor peso a vecinos inmediatamente adyacentes que a vecinos diagonales.

5.4.5. Restricciones de Crecimiento

Se implementan dos tipos de restricciones para realismo:

Restricciones absolutas Celdas con estado RESTRICTED ($U_{i,j} = 2$) no pueden urbanizarse:

$$P_{i,j} = 0$$

Restricción de tasa máxima En cada paso de simulación, el número de nuevas celdas urbanas N_{new} se limita a:

$$N_{new} \leq r_{max} \cdot N_{current} \quad (5.12)$$

donde $N_{current}$ es el número actual de celdas urbanas y r_{max} es la tasa máxima de crecimiento (típicamente 0,02–0,05, equivalente a un crecimiento del 2 al 5 % por iteración). Si el número de transiciones candidatas excede este límite, se seleccionan las celdas con mayor probabilidad.

5.4.6. Algoritmo de Simulación

Algorithm 2 Simulación AC-WoE con Decaimiento Exponencial

Input: Estado inicial U^0 , variables WoE $\{W_k\}$, parámetros θ , período T **Output:** Secuencia de estados $\{U^0, U^1, \dots, U^T\}$ $t = 0$ **to** $T - 1$ cada celda (i, j) no-urbana Calcular $P_{i,j}^{t+1}$ usando Ecuación ?? Generar número aleatorio $r \sim \mathcal{U}(0, 1)$ $r < P_{i,j}^{t+1}$ $U_{i,j}^{t+1} = 1$ (transición a urbano) $U_{i,j}^{t+1} = 0$ (permanece no-urbano) Aplicar restricciones post-procesamiento Actualizar métricas de monitoreo **return** $\{U^0, U^1, \dots, U^T\}$

5.5. Nota metodológica sobre la calibración

La calibración efectiva del modelo, la configuración final del Autómata Celular y el protocolo de validación quinquenal múltiple se presentan en el Capítulo ??, junto con las métricas que produce cada configuración. Para no separar el método del resultado, conviene dejar constancia aquí de cómo se obtuvieron los parámetros que aparecen en `data/processed/quinquenal_best_config.json`.

El **umbral de transición** ($\theta = 0,75$) y la **ponderación de las siete variables WoE** —proporcional al *Information Value* normalizado— se determinaron mediante búsqueda en cuadrícula y ajuste manual sobre el conjunto de entrenamiento (1984–2010), siguiendo el principio de adoptar la configuración más simple que mantuviera el FoM estable y evitara la sobre-predicción observada con umbrales menores.

Los **parámetros numéricos restantes** de la regla de transición —`neighbor_weight` y `distance_weight`— provienen de un experimento exploratorio previo y se adoptan en este trabajo como valores fijos. Su uso se justifica empíricamente por la estabilidad del modelo en las cinco ventanas de validación quinquenal: con esta misma configuración, sin reentrenar el WoE

ni reajustar dichos parámetros por período, el FoM se mantuvo en el intervalo $[0,222, 0,378]$ entre 2011 y 2020 (Capítulo ??). Esta estabilidad multitemporal es la evidencia que sostiene el uso de tales valores como constantes razonables del modelo. La calibración que esta tesis defiende, por tanto, es la exploración sistemática del umbral y de la ponderación por IV; el resto de los parámetros se trata como una elección operativa, documentada y reproducible, y no como una optimización metaheurística que excedería el alcance del trabajo.